REPUBIQUE DU SENEGAL

Un peuple Un But Une Foi

Ministère de l'Enseignement Supérieur de la Recherche et de l'Innovation (MESRI)

UNIVERSITE ALIOUNE DIOP DE BAMBEY

UFR : SCIENCES APPLIQUÉES ET TECHNOLOGIES DE L'INFORMATION ET DE LA COMMUNICATION(SATIC)

FILIERE : SYSTÈMES ET RÉSEAUX (SR)

NIVEAU : **MASTER 1**

Création d'une application Web dans un conteneur Docker

Présenté par :

Mohamadou Sabaly

Professeur :

Dr. Babou

Année académique 2025-2026

Introduction

Ce rapport présente la réalisation complète du TP portant sur la création d'une application Web déployée dans un conteneur Docker. L'objectif principal est de comprendre le cycle complet de développement : écriture d'un script Bash, développement d'une application Python/Flask, puis containerisation avec Docker.

Les étapes réalisées sont :

- Création d'un script Bash interactif
- Développement d'une application Web avec Flask (Python)
- Configuration des fichiers HTML/CSS de l'application
- Création d'un script Bash d'automatisation Docker
- Construction et exécution du conteneur Docker

Partie 2 : Création d'un script Bash simple

Cette partie introduit les bases du scripting Bash, indispensables pour automatiser des tâches dans les environnements DevOps et CI/CD.

Étapes réalisées

Étape 1-3 : Création du fichier et ajout du she-bang

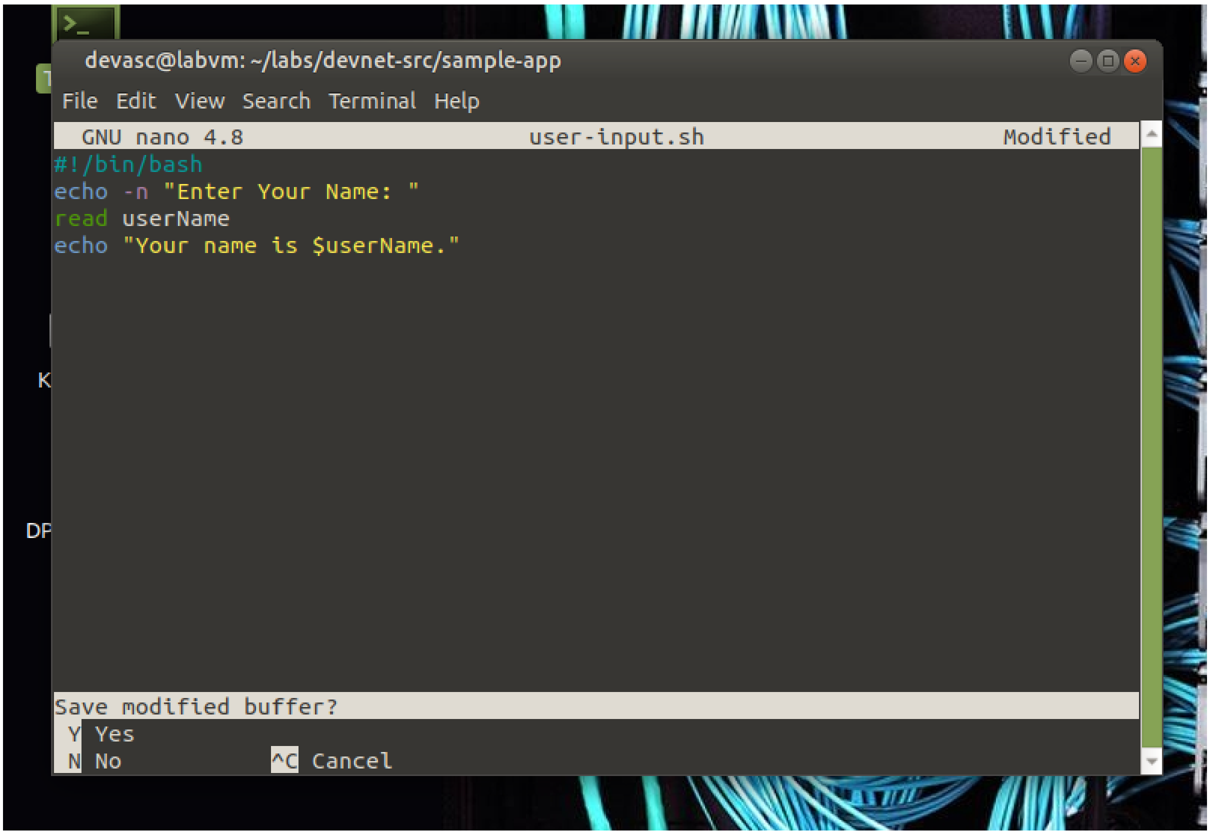
Changement de répertoire vers le dossier de travail et création du script :

```
cd ~/labs/devnet-src/sample-app
nano user-input.sh
```

Étape 4 : Ajout des commandes Bash

Le script demande le nom de l'utilisateur, le stocke dans une variable et l'affiche :

```
#!/bin/bash
echo -n "Enter Your Name: "
read userName
echo "Your name is $userName."
```



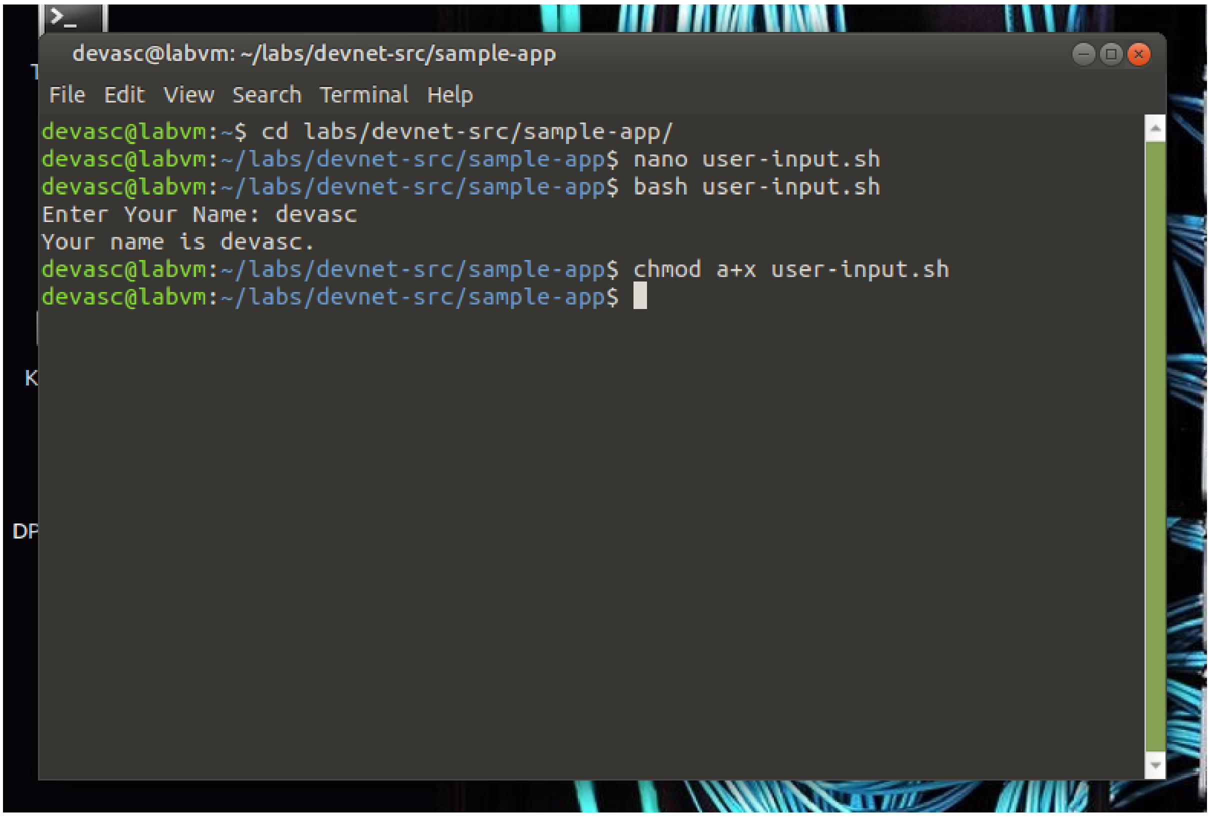
Étapes 6-7 : Exécution et permissions

Exécution directe via bash puis modification des droits d'exécution :

```
bash user-input.sh
chmod a+x user-input.sh
```

Résultat obtenu lors du test :

```
Enter Your Name: Ahmed Your name is decasc.
```

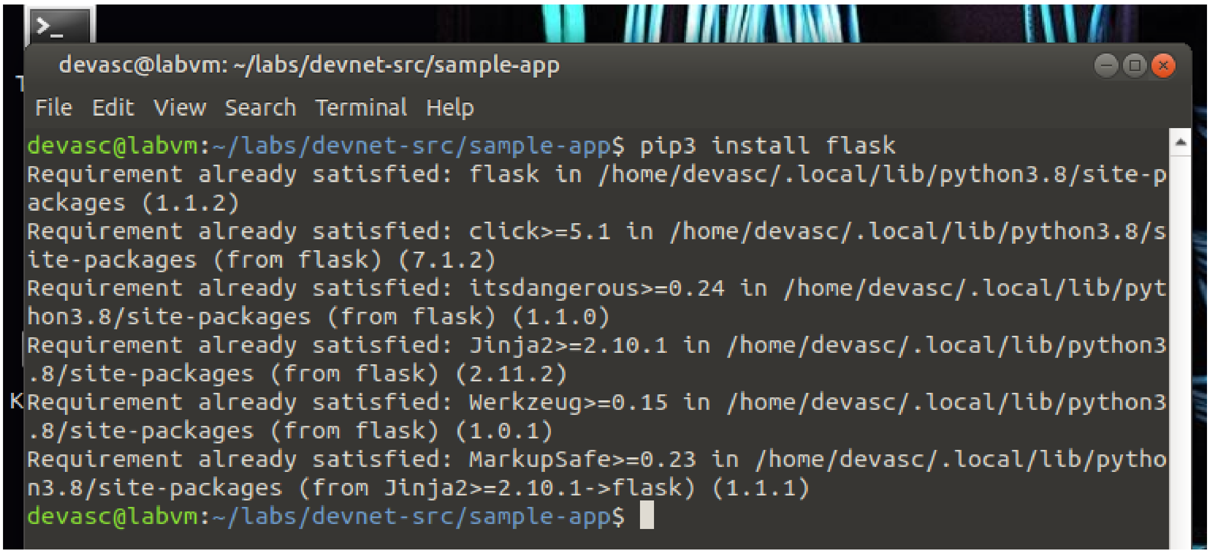


Partie 3 : Création de l'application Web avec Flask

Flask est un micro-framework Python pour développer des applications Web. Il permet de créer rapidement des routes HTTP et de gérer les requêtes/réponses.

Étape 1 : Installation de Flask

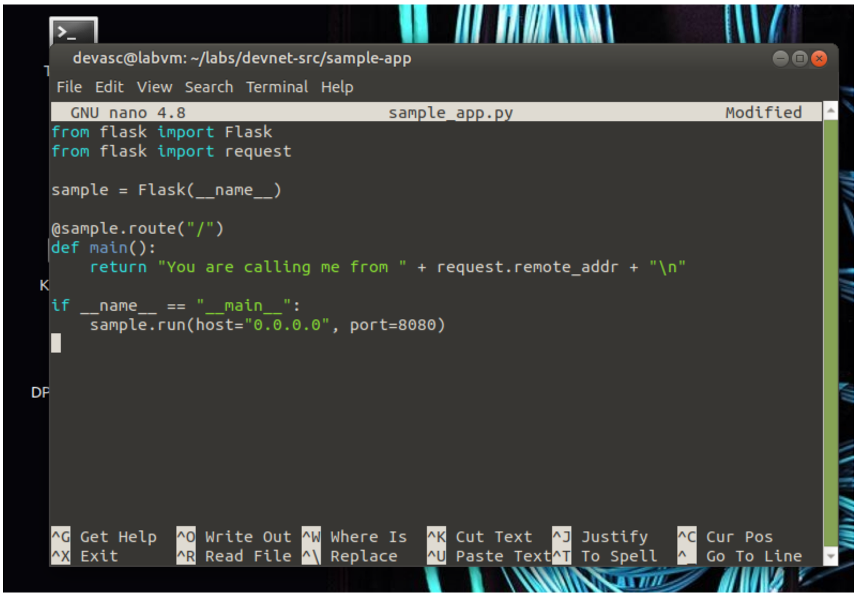
```
pip3 install flask
```



Étape 2-6 : Code de sample_app.py (version initiale)

Le fichier sample_app.py est créé avec le contenu suivant :

```
from flask import Flask
from flask import request

sample = Flask(__name__)

@sample.route("/")
def main():
    return "You are calling me from " + request.remote_addr + "\n"

if __name__ == "__main__":
    sample.run(host="0.0.0.0", port=8080)
```



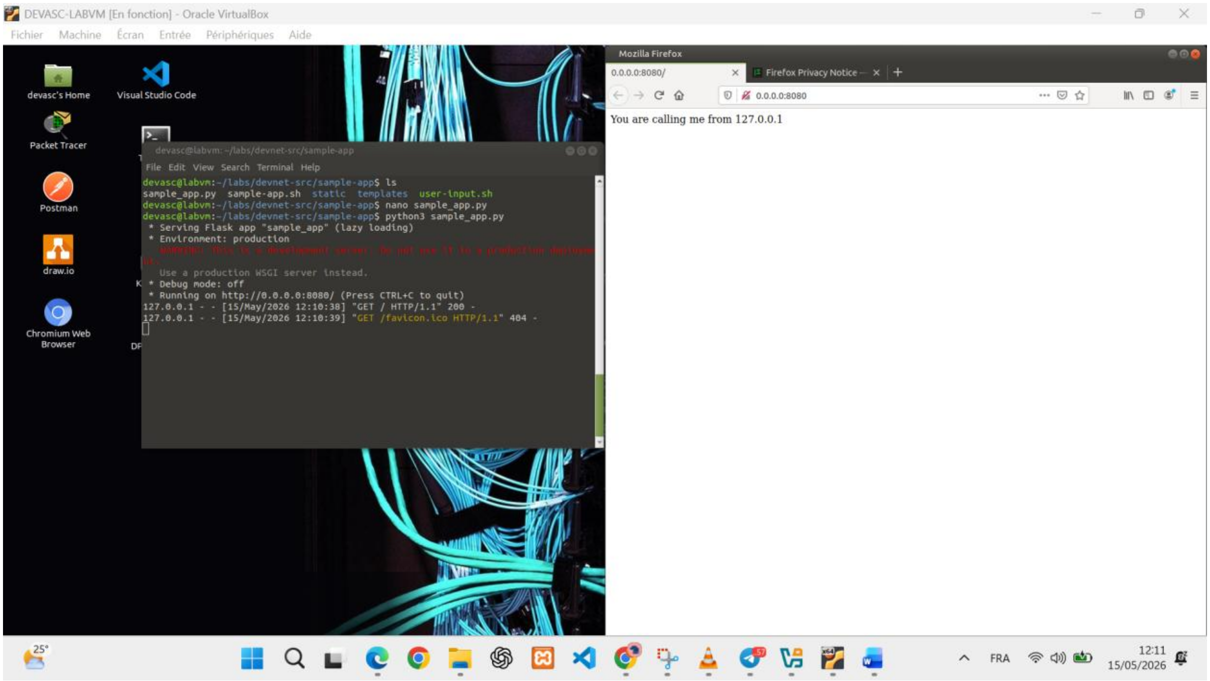
Étape 7-8 : Test du serveur

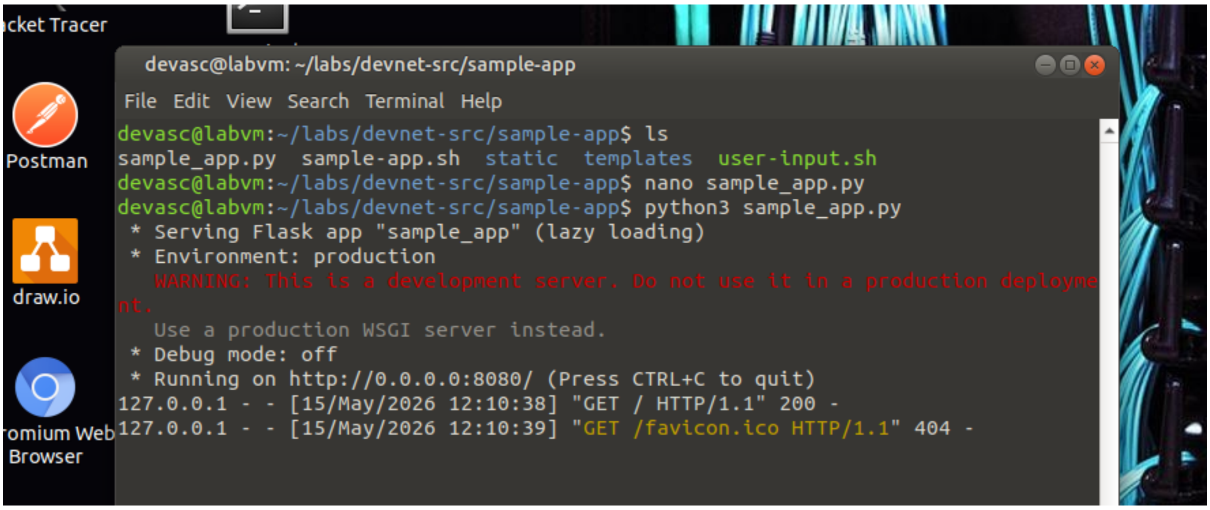
Lancement du serveur Flask :

```
python3 sample_app.py
* Serving Flask app "sample_app" * Running on http://0.0.0.0:8080/ Press CTRL+C to quit
```

Vérification avec cURL :

```
curl http://0.0.0.0:8080
You are calling me from 127.0.0.1
```





Partie 4 : Configuration avec fichiers HTML/CSS

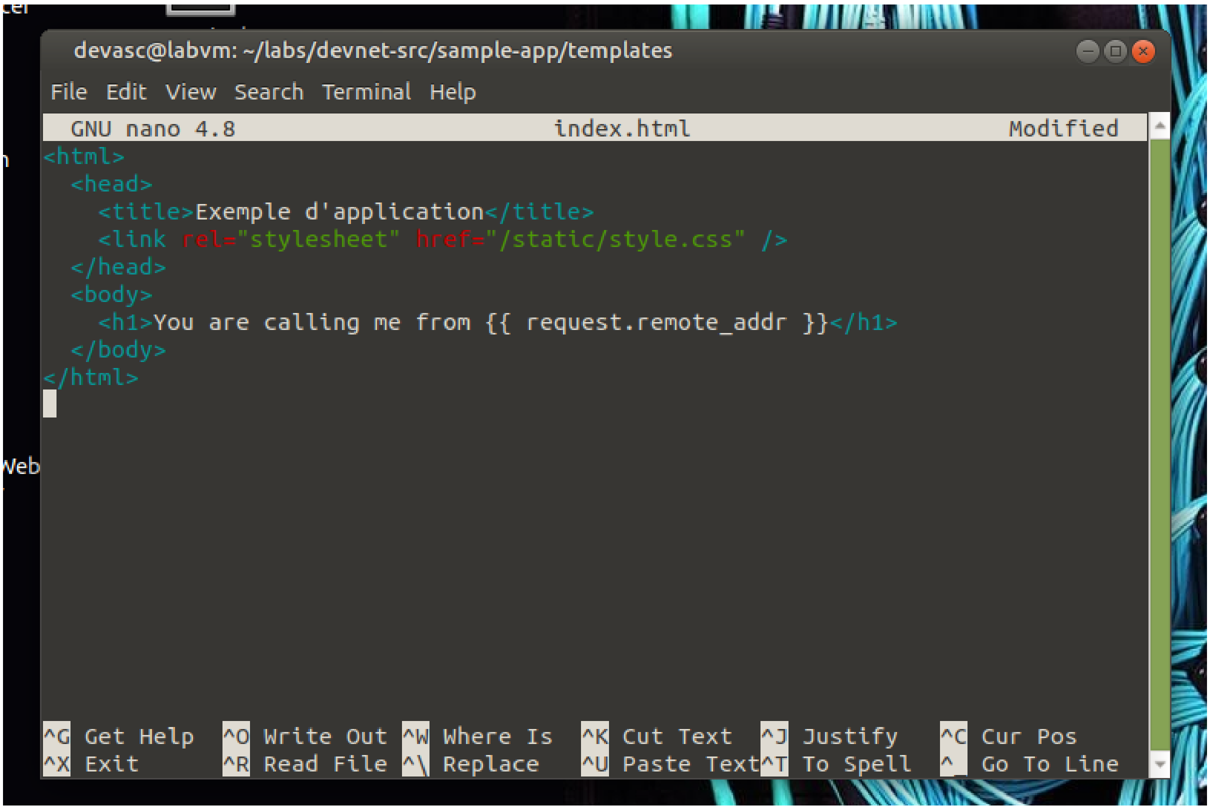
Dans cette partie, l'application est enrichie avec un fichier HTML (templates/index.html) et une feuille de style CSS (static/style.css).

Structure des répertoires

```
sample-app/
├── sample_app.py
├── templates/
│   └── index.html
└── static/
    └── style.css
```

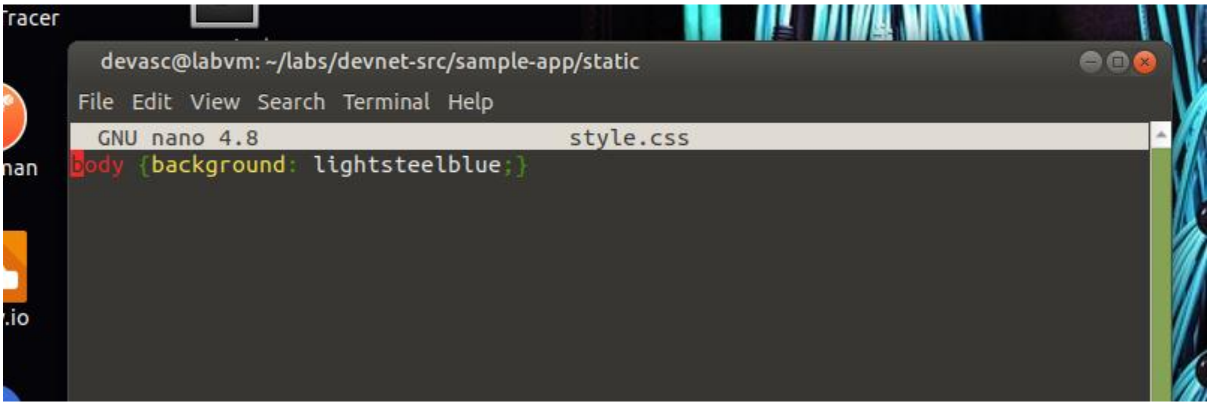
Contenu de templates/index.html

```
<html>
  <head>
    <title>Exemple d'application</title>
    <link rel="stylesheet" href="/static/style.css" />
  </head>
  <body>
    <h1>You are calling me from {{ request.remote_addr }}</h1>
  </body>
</html>
```



Contenu de static/style.css

```
body {background: lightsteelblue;}
```



Mise à jour de sample_app.py

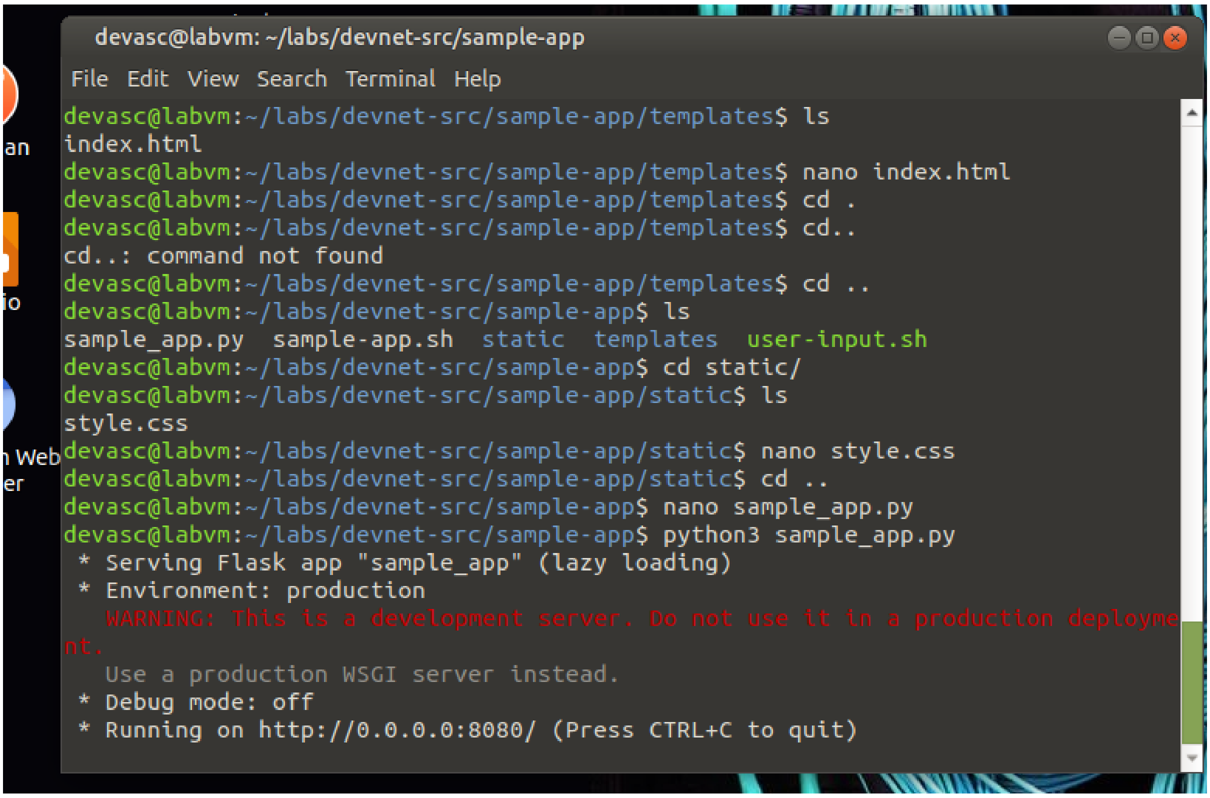
Ajout de render_template pour afficher le fichier HTML :

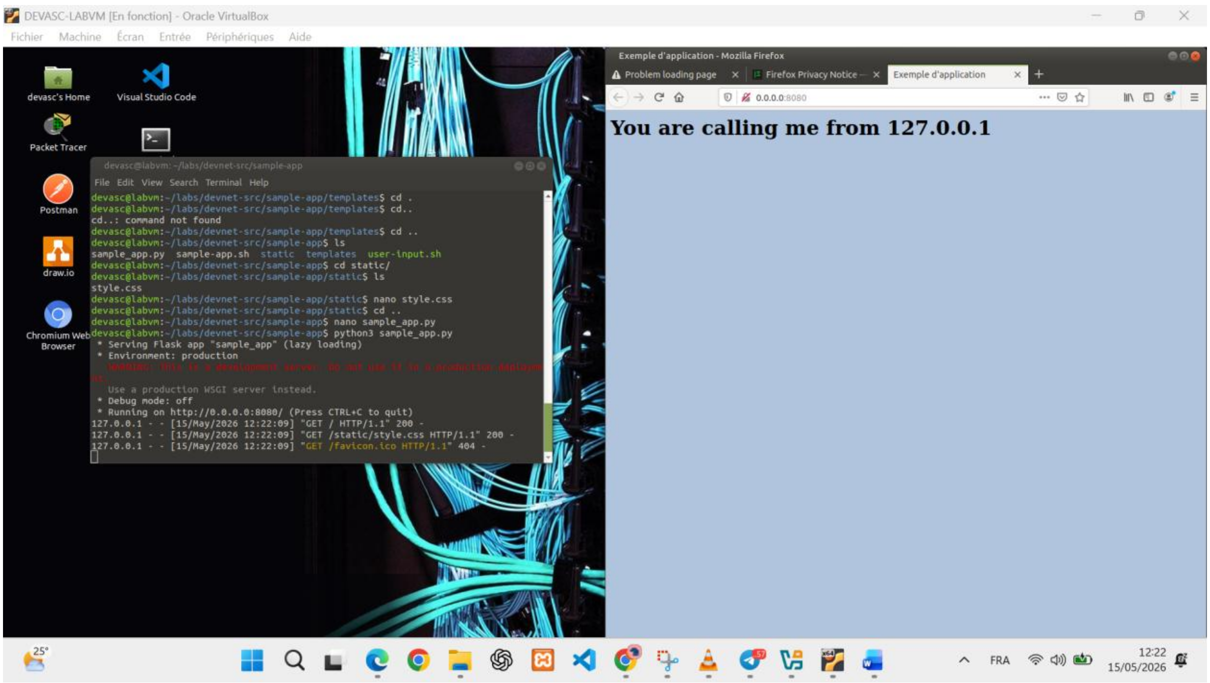
```
from flask import Flask
from flask import request
from flask import render_template

sample = Flask(__name__)

@sample.route("/")
def main():
    return render_template("index.html")

if __name__ == "__main__":
    sample.run(host="0.0.0.0", port=8080)
```

Vérification

Test avec cURL retournant le HTML complet :

```
curl http://0.0.0.0:8080
<html>   <head>     <title>Exemple d'application</title>   </head>   <body>
<h1>You are calling me from 127.0.0.1</h1>   </body> </html>
```



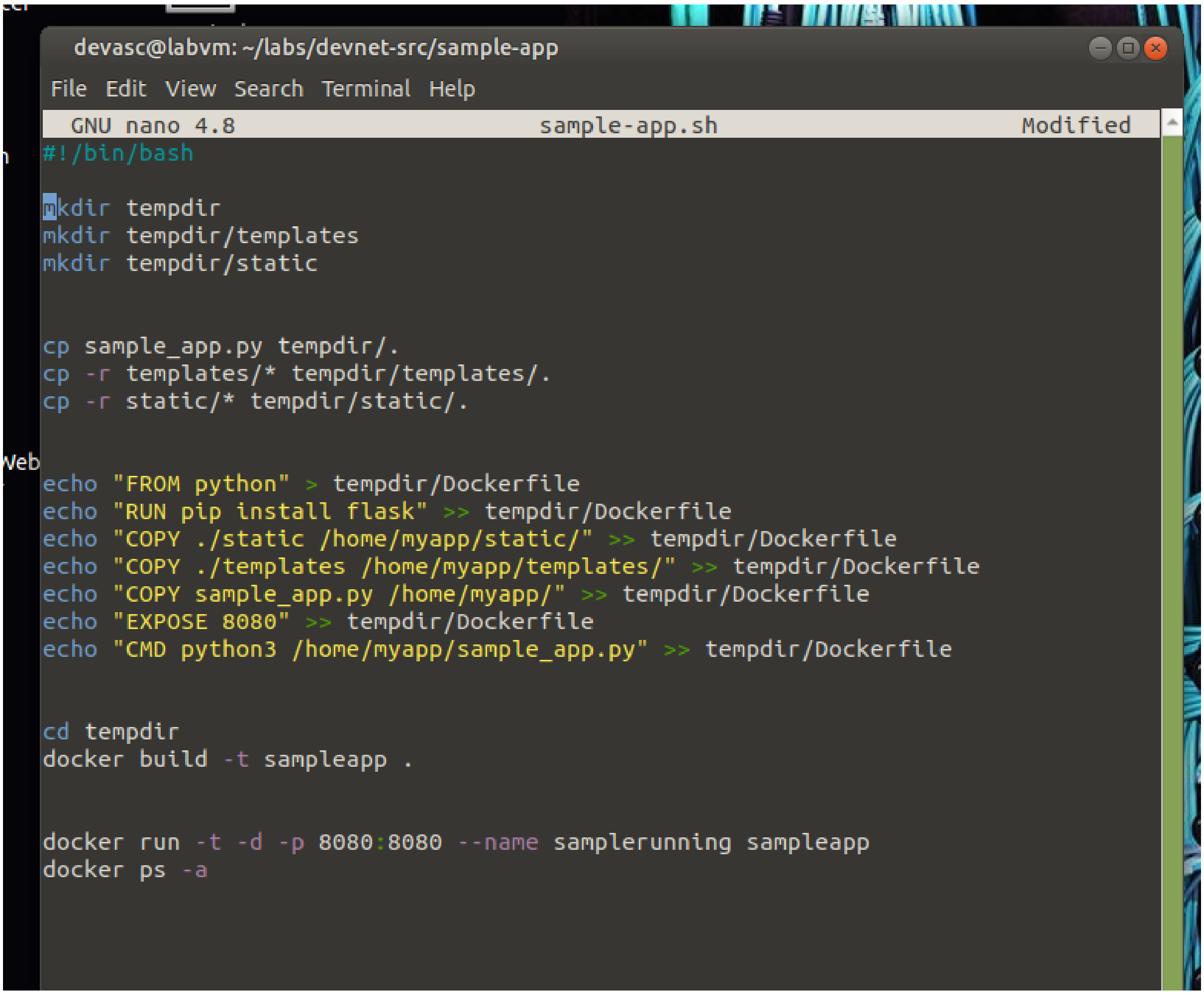


Partie 5 : Script Bash pour Docker

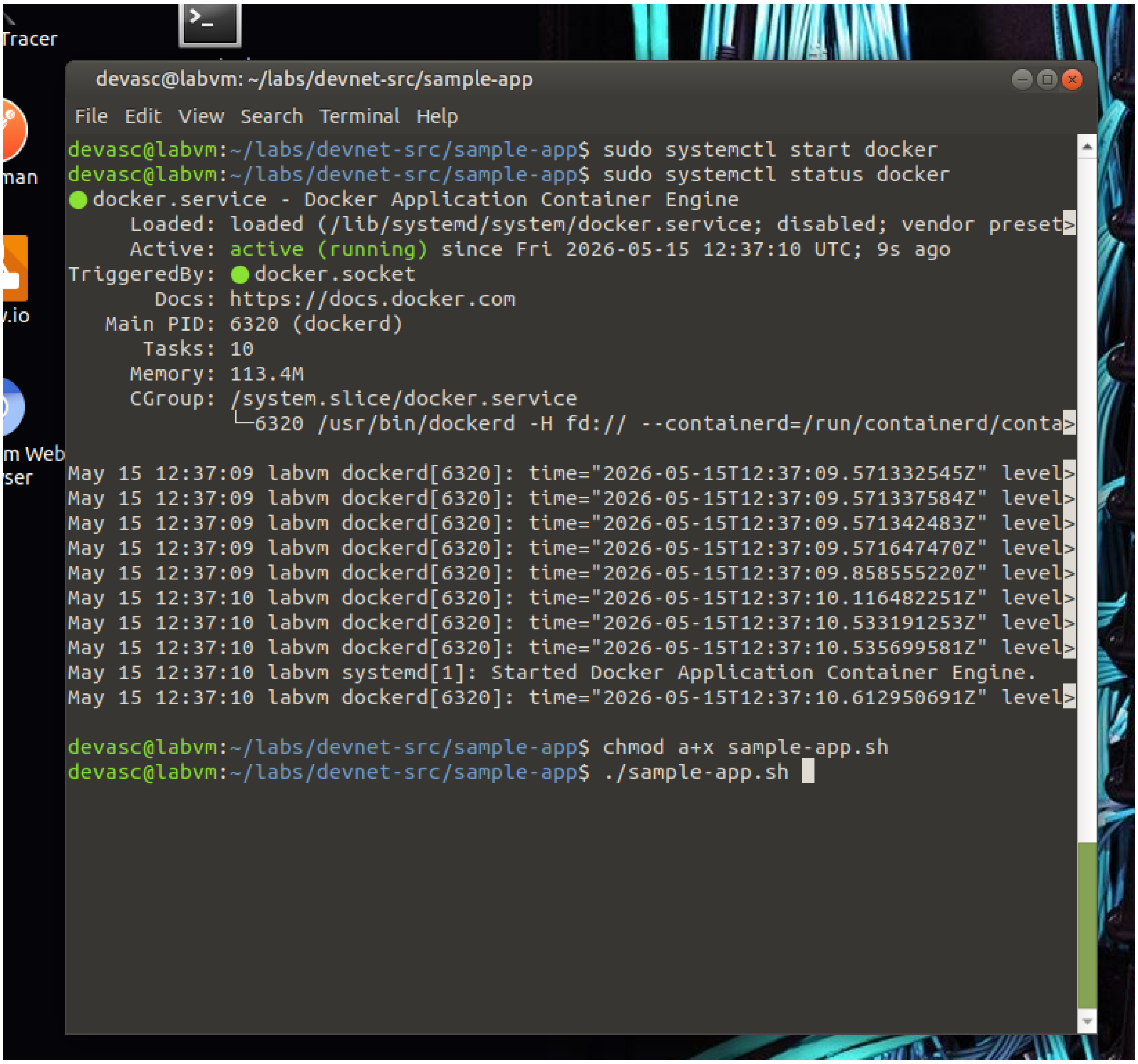
Ce script automatise entièrement la création et le lancement du conteneur Docker.

Contenu du fichier sample-app.sh

```
#!/bin/bash

# Étape 1 : Créer les répertoires temporaires
mkdir tempdir
mkdir tempdir/templates
mkdir tempdir/static

# Étape 2 : Copier les fichiers
cp sample_app.py tempdir/.
cp -r templates/* tempdir/templates/.
cp -r static/* tempdir/static/.

# Étape 3 : Créer le Dockerfile
echo "FROM python" > tempdir/Dockerfile
echo "RUN pip install flask" >> tempdir/Dockerfile
echo "COPY ./static /home/myapp/static/" >> tempdir/Dockerfile
echo "COPY ./templates /home/myapp/templates/" >> tempdir/Dockerfile
echo "COPY sample_app.py /home/myapp/" >> tempdir/Dockerfile
echo "EXPOSE 8080" >> tempdir/Dockerfile
echo "CMD python3 /home/myapp/sample_app.py" >> tempdir/Dockerfile

# Étape 4 : Construire le conteneur Docker
cd tempdir
docker build -t sampleapp .

# Étape 5 : Démarrer et vérifier
docker run -t -d -p 8080:8080 --name samplerunning sampleapp
docker ps -a
```


devasc@labvm: ~/labs/devnet-src/sample-app
File Edit View Search Terminal Help
GNU nano 4.8
sample-app.sh
Modified
#!/bin/bash
mkdir tempdir
mkdir tempdir/templates
mkdir tempdir/static
cp sample_app.py tempdir/.
cp -r templates/* tempdir/templates/.
cp -r static/* tempdir/static/.
echo "FROM python" > tempdir/Dockerfile
echo "RUN pip install flask" >> tempdir/Dockerfile
echo "COPY ./static /home/myapp/static/" >> tempdir/Dockerfile
echo "COPY ./templates /home/myapp/templates/" >> tempdir/Dockerfile
echo "COPY sample_app.py /home/myapp/" >> tempdir/Dockerfile
echo "EXPOSE 8080" >> tempdir/Dockerfile
echo "CMD python3 /home/myapp/sample_app.py" >> tempdir/Dockerfile
cd tempdir
docker build -t sampleapp .
docker run -t -d -p 8080:8080 --name samplerunning sampleapp
docker ps -a

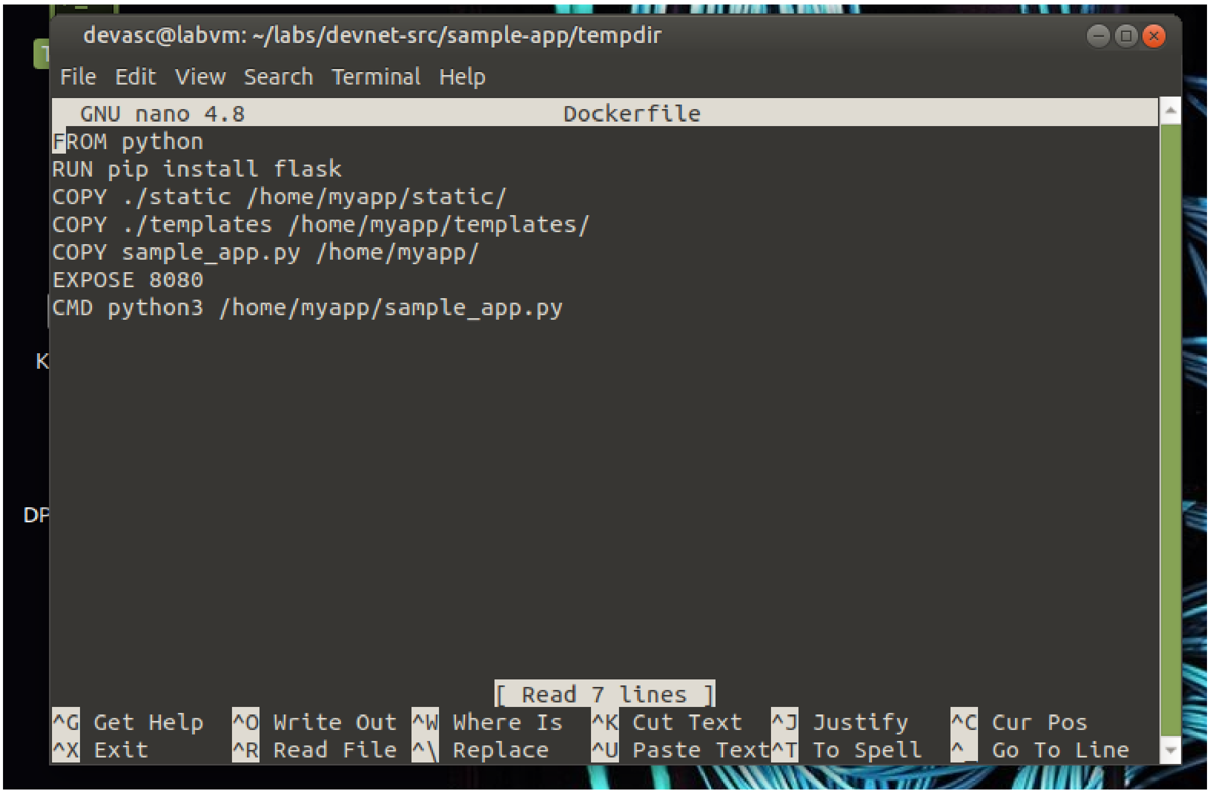
Vérifier si docker est allumer



Le Dockerfile généré

Voici le contenu du fichier tempdir/Dockerfile créé par le script :

```
FROM python
RUN pip install flask
COPY ./static /home/myapp/static/
COPY ./templates /home/myapp/templates/
COPY sample_app.py /home/myapp/
EXPOSE 8080
CMD python3 /home/myapp/sample_app.py
```



Partie 6 : Build, exécution et vérification

Je nai pas pu executer le script et telecharge tous les bilio par probleme de connexion